



## Tema 18.4: Agentes prototyping

# Contenidos

- Fundamentos Agentes
- Componentes Agentes
- Orquestación multi-agente
- Workshop Agentes
- Reto
- Agentes en producción



# Fundamentos Agentes



Las plataformas no-code te llevan a una demo en minutos. **El código te lleva a producción.**

## **Agentes no-code**

- Camino más rápido a una demo funcional
- Flujo visual, fácil para no-desarrolladores
- Control limitado sobre prompts y tools
- Difícil de testear, evaluar y versionar
- Vendor lock-in (UI, runtime, facturación)
- Difícil de integrar a escala

## **Agentes code-first**

- **Control total** sobre prompts, tools y memoria
- Versionado en git, revisable en PRs
- Testeable unitariamente, apto para evals
- Sin vendor lock-in
- Observabilidad de producción
- Integrable con cualquier stack de backend

**Estrategia híbrida:** Usa no-code para validar la idea · cambia a código en cuanto necesites control, evals o producción

Los candidatos code-first serios para prototipar agentes en 2026.

## OpenAI Agents SDK

Curva: **fácil** · Python / TS

Ideal: handoffs, flujos multi-agente sobre stack OpenAI

## Pydantic AI

Curva: **fácil** · Python

Ideal: agentes type-safe, I/O estructurada, model-agnostic

## LangGraph

Curva: **media** · Python / TS

Ideal: grafos explícitos, workflows complejos con estado

## CrewAI

Curva: **fácil** · Python

Ideal: crews por roles, procesos secuenciales/jerárquicos

## Microsoft Agent Framework

Curva: **media** · .NET / Python

Ideal: agentes enterprise nativos en Azure (Semantic Kernel + AutoGen)

## Agno

Curva: **fácil** · Python

Ideal: agentes ultrarrápidos con memoria y RAG integrados

Todos open source · todos model-agnostic · elige según skills del equipo y necesidades de integración.



# ¿Qué framework elegir?

Ninguno está mal. Elige el que encaje con tu escenario.

## Prototipo rápido (1 agente)

**Pydantic AI** → Type-safe y mínimo boilerplate. La mejor opción por defecto para nuevos proyectos Python.

## Ecosistema OpenAI

**OpenAI Agents SDK** → Handoffs nativos y trazabilidad total en el dashboard de OpenAI con mínimo esfuerzo.

## Workflows con estado

**LangGraph** → Grafos explícitos, checkpoints y "human-in-the-loop" para procesos complejos y duraderos.

## Multi-agente por roles

**CrewAI** → Roles declarativos y orquestación (secuencial o jerárquica) para equipos de agentes.

Hoy nos centramos en OpenAI Agents SDK · Pydantic AI como alternativa type-safe · los patrones aplican a todos.

**LangChain / LangGraph /  
LangSmith**



# LangChain: el framework que popularizo los LLMs

## Que es

- Framework open-source para construir aplicaciones LLM
- con componentes reutilizables y 100+ integraciones.
- Prompt + Modelo + Parser + Memoria + Tools -- todo bajo una API comun y componible.

## LCEL (LangChain Expression Language)

- Operador | encadena bloques:
- prompt | llm | parser
- Lazy: no ejecuta hasta .invoke().
- Composable, serializable y testeable nodo a nodo.

## Cuando elegirlo

- Necesitas RAG, agentes con tools o pipelines batch.
- Quieres una base solida para escalar a LangGraph.
- Ecosystem: Chroma, Pinecone, FAISS, PDF loaders...

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```
llm = ChatOpenAI(model='gpt-4o-mini')
prompt = ChatPromptTemplate.from_template('Explica {tema} en 2 frases')
parser = StrOutputParser()
```

```
cadena = prompt | llm | parser      # LCEL
resultado = cadena.invoke({'tema': 'RAG'})
print(resultado) # string limpio, sin parsear
```

## Gratis con LangChain

- 100+ LLM providers listos
- Vectorstores: Chroma, FAISS, Pinecone, pgvector
- Loaders: PDF, CSV, web, Notion, YouTube
- ConversationChain, RetrievalQA, RAG listos



Tres patrones que cubren el 80% de los casos en producción.

## Salida estructurada (Pydantic)

```
class Sentiment(BaseModel):
    score: float # 0.0 a 1.0
    label: str # positivo/negativo

llm_s =
llm.with_structured_output(Sentiment)
chain = prompt | llm_s
r = chain.invoke({'texto': texto})
print(r.score, r.label) # tipado, sin json.loads
```

## Pipeline multi-paso

```
resumir = prompt_r | llm | parser
traducir = prompt_t | llm | parser

pipeline = (
    resumir
    | RunnableLambda(
        lambda t: {'texto': t})
    | traducir
)
res = pipeline.invoke({'texto': doc})
```

## Batch y Streaming

```
# Batch: inputs en paralelo
resultados = chain.batch([
    {'tema': 'RAG'},
    {'tema': 'Agentes'},
])

# Streaming: token a token
for chunk in chain.stream(
    {'tema': 'LangGraph'}):
    print(chunk, end="", flush=True)
```

Todos los patrones son composables: puedes meter una cadena dentro de otra como si fuera un bloque más.

## Cuando usar cada patron

Structured output: cuando el código downstream necesita campos tipados (BDs, APIs, dashboards).

Multi-paso: cuando la salida de un LLM es el input de otro (resumir + traducir, extraer + validar).

Batch: para procesar cientos de items en paralelo. Streaming: UIs de chat donde el usuario ve tokens en tiempo real.

# Agentes con herramientas en LangChain

@tool convierte cualquier función Python en algo que el LLM puede decidir invocar.

```
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
```

## @tool

```
def buscar_precio(ticker: str) -> str:
    """Devuelve el precio de la acción dado su ticker (ej: AAPL)."""
    precios = {'AAPL': 213.5, 'MSFT': 415.8, 'NVDA': 118.3}
    return f"{ticker}: ${precios.get(ticker.upper(), 'N/A')}"
```

## @tool

```
def calcular_cambio(p_ini: float, p_fin: float) -> str:
    """Calcula el porcentaje de cambio entre dos precios."""
    cambio = ((p_fin - p_ini) / p_ini) * 100
    return f"Cambio: {cambio:+.2f}%"
```

```
agente = create_tool_calling_agent(llm, [buscar_precio, calcular_cambio], prompt)
executor = AgentExecutor(agent=agente, tools=[buscar_precio, calcular_cambio],
                          verbose=True) # muestra el ciclo en consola
```

```
resultado = executor.invoke({'input': 'Cuanto vale AAPL y cuanto ha subido desde 195?'})
```

## Cómo decide el agente

El LLM lee el docstring + type hints de la tool y decide si usarla, cuando y con qué argumentos. Buen docstring = tool bien usada.

## El loop ReAct

Razonar -> Invocar tool -> Observar resultado -> Razonar de nuevo. Se repite hasta tener respuesta final sin más tool calls.

## verbose y LangSmith

verbose=True en dev. En prod, LangSmith traza cada ciclo con latencia, tokens y tool calls en una UI navegable.

Regla: docstrings buenos = el modelo usa las tools correctamente. Trata el docstring como el contrato entre tu código y el LLM.

# LangGraph: estado explícito y flujos con control total

LangChain encadena linealmente. LangGraph programa grafos donde el estado persiste y las ramas son explícitas.

## Estado (State)

TypedDict compartido por todos los nodos.

Cada nodo lee el estado, trabaja y escribe de vuelta.

Ningun nodo necesita conocer a los demas.

## Nodos (Nodes)

Funciones Python: reciben el estado, devuelven un dict.

Pueden llamar LLMs, tools, APIs o cualquier código.

Se testean de forma aislada -- solo necesitan el estado.

## Edges condicionales

Una funcion de enrutamiento lee el estado

y devuelve el nombre del siguiente nodo.

Aqui viven las decisiones, bucles y ramas del agente.

## Usa LangGraph cuando...

El flujo tiene ramas o bucles que dependen de datos en tiempo de ejecución.

Necesitas pausar y esperar aprobación humana (HITL).

Quieres checkpoints para reiniciar si algo falla.

## Queda en LangChain cuando...

El flujo es lineal: A -> B -> C sin ramas ni retrocesos.

Agente simple: un LLM con tools sin estado complejo.

Prototipado rapido y aun no necesitas control fino.

LangGraph esta construido SOBRE LangChain -- no son alternativas. Usas LangGraph cuando LCEL ya no es suficiente.

# LangGraph: Human-in-the-Loop y Checkpoints

El grafo puede pausarse entre dos nodos y reanudarse horas después con el estado intacto.

1. Usuario pide acción

2. Agente analiza y propone

3. **PAUSA -- Espera aprobación humana**

4. Humano aprueba o rechaza

5. Grafo se reanuda y ejecuta

```
from langgraph.checkpoint.memory import MemorySaver

# MemorySaver guarda el estado entre invocaciones
# En prod: SqliteSaver, PostgresSaver
checkpointer = MemorySaver()

app = grafo.compile(
    checkpointer=checkpointer,
    interrupt_after=['analizar'] # PAUSA aqui
)

config = {'configurable': {'thread_id': 'orden-001'}}

# Paso 1: ejecutar hasta la pausa
estado = app.invoke(input_inicial, config=config)
# >>> GRAFO PAUSADO -- el humano revisa la propuesta <<<

# Paso 2: el humano modifica el estado directamente
app.update_state(config, {'aprobada': True})

# Paso 3: reanudar desde el checkpoint guardado
resultado = app.invoke(None, config=config) # None = continuar
```

## Casos de uso reales de Human-in-the-Loop

Aprobar un email antes de enviarlo, Validar una compra o transferencia, Confirmar borrado de datos sensibles...

# LangSmith: observabilidad y evaluacion para agentes

## Trazas

- Cada llamada al LLM, cada tool call, cada nodo del grafo.
- Capturado como span. Ve exactamente qué ocurrió.
- En qué orden y cuanto tiempo tardo cada paso.

## Coste y tokens

- Tokens in/out por traza, por sesion, por usuario.
- Identifica que prompts o flujos consumen más
- y optimizarlos antes de que llegue la factura.

## Evals automaticos

- Define criterios: correctness, relevancia, toxicidad.
- Corre evals contra un dataset cuando cambias
- el prompt o el modelo -- compara antes/después.

## Datasets y replay

- Guarda ejemplos reales como dataset.
- Re-ejecuta sesiones pasadas con un prompt nuevo
- y compara salidas lado a lado.

```
# Solo estas 3 variables activan LangSmith -- sin cambiar tu codigo
import os
os.environ['LANGSMITH_API_KEY'] = 'lsv2_...'
os.environ['LANGSMITH_TRACING'] = 'true'
os.environ['LANGSMITH_PROJECT'] = 'mi-proyecto'
```

```
# A partir de aqui toda llamada LangChain/LangGraph
# aparece en smith.langchain.com automaticamente
```

**smith.langchain.com**

Free: 5,000 trazas/mes  
Developer: ilimitado uso personal

Alternativas OSS: Langfuse, Phoenix,  
OpenTelemetry.

# El stack: LangChain + LangGraph + LangSmith

## LangSmith -- OBSERVABILIDAD

- Trazas, costes, evals, datasets. La capa de visibilidad.
- Se activa con 3 variables de entorno, sin cambiar código.
- Recomendado desde el primer día, no solo en producción.

## LangGraph -- ORQUESTACION

- Grafos con estado, ramas condicionales, bucles...
- Se usa cuando una cadena LCEL ya no es suficiente.
- Construido sobre LangChain -- son capas, no alternativas.

## LangChain -- FUNDAMENTO

- LCEL, prompts, modelos, parsers, tools, memoria.
- La base sobre la que corren los agentes y los grafos.
- 100+ integraciones listas: LLMs, vectorstores, loaders.



# Workshop: construye tu agente LangGraph

## Reto: Agente clasificador con estado y herramientas

Construye un agente LangGraph que reciba una pregunta, la clasifique, llame a la herramienta correcta y responda.

### 1. Define el Estado

TypedDict con messages (add\_messages), ruta (str) y al menos un campo más. Piensa que datos necesita cada nodo.

### 2. Crea las herramientas

Al menos 2 funciones con @tool. Docstrings claros: el LLM los usa para decidir cuando invocarlas.

### 3. Nodo clasificador

Un nodo que llama al LLM para decidir la ruta según la pregunta del usuario. Escribe el resultado en state['ruta'].

### 4. Nodos de respuesta

Un nodo por ruta. Cada uno usa su tool correspondiente y genera un AIMessage con la respuesta.

### 5. Edge condicional

Función enrutar() que lee state['ruta'] y devuelve el nombre del siguiente nodo. Regístrala con add\_conditional\_edges.

### 6. Compila, ejecuta y observa

app = grafo.compile(). Prueba con 3 preguntas distintas. Verifica las trazas en LangSmith para ver el ciclo completo.

## OpenAI Agents SDK



SDK Python ligero de OpenAI. Cinco primitivas: Agents, Tools, Handoffs, Guardrails, Sessions.

## Qué te da

Decoradores `@function_tool` · tracing automático · handoffs integrados · primitiva de guardrails

## Ideal para

Equipos ya en el stack OpenAI que quieren mínimas abstracciones y handoffs llave en mano.

```
# hello_agent.py
from agents import Agent, Runner,
function_tool @function_tool

def get_weather(city: str) ->
    str: return f"En {city} hace
    sol."

agent = Agent(name="Assistant", instructions="Help with weather",
tools=[get_weather]) result = Runner.run_sync(agent, "¿Qué tiempo hace en Madrid?" )
print(result.final_output)
```

## Componentes Agentes



# Llamada a herramientas

Tools = funciones Python que el agente puede llamar.

El modelo decide cuándo, con qué argumentos, y lee el resultado.

```
from agents import Agent,  
function_tool  
  
@function_tool  
def get_user (user_id: int) -> dict:  
    """Consulta un usuario en la  
    BD.""" return  
    db.fetch_user(user_id)  
  
agent = Agent(  
    name='Support'  
)  
,  
    instructions= 'Ayuda con usuarios'  
, tools=[get_user],
```

**Decorador + type hints + docstring** → el modelo recibe un schema limpio gratis.

# ¿Qué puede ser una tool?



## Búsqueda y recuperación

Consulta a vector store, búsqueda web, lookup en KB interna



## Base de datos

Consultas SQL, lookups ORM, lecturas key-value



## APIs externas

Llamadas REST, tiempo, pagos, servicios internos



## Operaciones con ficheros

Leer PDF, parsear hoja de cálculo, escribir informe, gestionar uploads



## Ejecución de código

Ejecutar Python en sandbox, ejecutar SQL, evaluar fórmulas



## Otros agentes

Tool = llamada a un sub-agente. Base de la orquestación.

**Regla general:** tools pequeñas, deterministas y bien nombradas ganan a una mega-tool.

Capa de validación bidireccional para garantizar la robustez y seguridad del agente.

## Guardarraíles de entrada (Input)

### Filtro preventivo antes del LLM

- **Detección off-topic:** PII, prompt injection.
- **Filtro de contenido:** Material dañino o regulado.
- **Validación de schema:** Campos y longitudes.
- **Rate limiting:** Control de costes por usuario.

## Guardarraíles de salida (Output)

### Verificación post-generación

- **Salida estructurada:** Validación vía Pydantic.
- **Citas y fuentes:** Verificación de veracidad.
- **Brand safety:** Filtro de toxicidad y tono.
- **Tool call sanity:** Control de llamadas a tablas.

**Fail-fast:** el agente reintenta o escala ante fallos críticos, evitando respuestas degradadas.

# Guardarraíles en la práctica

Agents SDK: los validators lanzan ValueError. Combinable con output\_type=Pydantic para schema fuerte.

```
agent_with_guardrails.py
from agents import (Agent, output_guardrail,
                    GuardrailFunctionOutput)
from pydantic import BaseModel

class Answer(BaseModel):
    texto: str
    citas: list[str]

@output_guardrail
async def exigir_citas(ctx, agent, out):
    if not out.citas:
        raise ValueError('Falta cita')
    return GuardrailFunctionOutput('', False)
```

La validación vive en el código, no en el prompt · reproducible, testeable, versionada.



Tres capas, tres stores distintos. No metas todo en el prompt.

## Corto plazo

### Historial de conversación

- Vive en la lista de mensajes pasada al modelo.
- Conserva los últimos N turnos o resume los más antiguos.
- Storage: dict en memoria, Redis, helper del framework.

## Largo plazo

### Semántica / vectorial

- Embeddings de notas, docs y hechos pasados.
- Recuperados on-demand vía tool de búsqueda.
- Storage: Chroma, Qdrant, pgvector, Pinecone.

## Estructurada

### Hechos y perfiles

- Perfil de usuario, preferencias, entidades, transacciones.
- Consultados como tool, no metidos en el prompt.
- Storage: Postgres, MongoDB, tu BD existente.

Elige una capa por tipo de memoria. Mezclarlas en un prompt = drift, coste y bugs.

Agents SDK: pasa session=SQLiteSession(...) entre turnos. Memoria a largo plazo vía una tool que consulta un vector store.

## Implementación

### Corto plazo

Se gestiona mediante **SQLiteSession** para mantener el estado entre turnos.

### Largo plazo

Implementada como una **tool** que realiza consultas semánticas a un vector store.

```
# chat_with_memory.py
from agents import Agent, Runner, SQLiteSession

agente = Agent(
    name='Mentor',
    instructions='...'
)
sesion = SQLiteSession('user_001')

for msg in conversación: res
    = Runner.run_sync( agente,
    msg, session=sesion
    )
print(res.final_output)
```

Corto plazo = **SQLiteSession** · Largo plazo = **tool de consulta vectorial**.

Los modelos Pydantic = el contrato entre el agente y tu código.



## Por qué importa

- ✓ Sin parsear texto LLM con regex.
- ✓ Comprobación de tipos en runtime.
- ✓ Integración directa con sistemas downstream – BDs, APIs, colas.

```
# invoice_agent.py

from agents import Agent, Runner
from pydantic import BaseModel

class Invoice(BaseModel):
    invoice_id: str
    total: float

agent = Agent(
    name='Invoice',
    output_type=Invoice,
)
res = Runner.run_sync(agent, texto)
print(res.final_output.total)
```

La validación vive en el **schema**, no en el prompt.

Salida token a token para UX. I/O async para tools concurrentes.

## Streaming

Muestra la salida parcial mientras el modelo piensa. Crítico para UIs de chat y respuestas largas.

```
async for evt in Runner.run_streamed(  
    agent, prompt  
).stream_events(): ...
```

## Tools async

Las llamadas a tools devuelven en paralelo. Ejecuta 3 búsquedas web a la vez sin bloquear.

```
@function_tool  
async def search(...):  
    await httpx.get(...)
```

No puedes mejorar un agente que no puedes ver.



## Trazas

Cada llamada al modelo, cada tool call, cada retry — capturado como span.



## Coste

Tokens in/out por turno, por sesión, por usuario.



## Errores

Tool calls fallidas, reintentos de validators, alucinaciones detectadas.



## Replay

Re-ejecuta una sesión con prompt o modelo nuevos — diferencia las salidas.

Herramientas: Logfire · OpenTelemetry · Langfuse · Phoenix · dashboard de trazas de OpenAI.

# Recap de componentes

Seis bloques. Domínalos y construyes el 80% de agentes útiles.

## Herramientas

Funciones Python tipadas.  
Pequeñas y deterministas.

## Guardarraíles

Valida entrada y salida en  
código.

## Memoria

Tres capas, tres stores.  
No al prompt-stuffing.

## Salida estructurada

Pydantic = contrato  
agente↔código.

## Streaming

UX de chat fluida. Async para  
tools concurrentes.

## Observabilidad

Trazas, costes, errores, replay.

## Orquestación multi-agente



Cuatro patrones que cubren el 90% de los sistemas multi-agente.

## Pipeline secuencial

Agente 1 → Agente 2 → Agente 3. Cada uno añade una transformación.

**Ideal para:** flujos tipo ETL, producción de contenido.

## Router

Un único dispatcher enruta a uno de N especialistas según la intención.

**Ideal para:** soporte al cliente, QsA multi-dominio.

## Supervisor / jerarquía

Un supervisor delega subtareas a workers y replanifica.

**Ideal para:** investigación abierta, planificación.

## Colaborativo

Agentes pares debaten o votan.

**Ideal para:** decisiones críticas, code review, validación adversarial.

Elige el patrón más pequeño que resuelva el problema.



OpenAI Agents SDK ofrece handoffs de manera primitiva. El agente actual cede el control a otro.

## Cuándo usarlo

- Delegación entre dominios distintos (billing vs técnico).
- Sub-agente con system prompt o modelo distinto.
- UI o tracing necesitan saber «ahora habla otro».

handoffs.py

```
from agents import Agent, Runner

billing = Agent(name="Billing", instructions="...")
refunds = Agent(name="Refunds", instructions="...")

triage = Agent(
    name="Triage",
    instructions="Route the user to the right
) agent.", handoffs=[billing, refunds],

result = await Runner.run(triage, "Quiero un reembolso")
```

Handoff = transferencia total de control. Sub-agente = tool call que devuelve un valor.

Modela equipos como crews: cada agente con rol y goal, encadenados por tasks tipadas.

## Modelo mental

- Agent = persona con rol + goal.
- Task = unidad de trabajo con expected\_output.
- Crew = orquestación (sequential / hierarchical) sobre los agentes.

crew.py

```
from crewai import Agent, Task, Crew, Process

researcher = Agent(role="Investigador de mercado",
                    goal="Encontrar datos de competidores", ...)
writer = Agent(role="Redactor",
               goal="Escribir un informe de 1 página", ...)

research = Task(description="Investiga el SKU X",
                 expected_output="lista de bullets", agent=researcher)
write = Task(description="Redacta informe",
              expected_output="markdown", agent=writer,
              context=[research])

Crew(agents=[researcher, writer], tasks=[research, write],
     process=Process.sequential).kickoff()
```

Ideal para pipelines de contenido. Menos para chat de baja latencia.

El estado compartido decide cómo se entienden tus agentes. Tres opciones, en orden de fricción.

## Historial de mensajes

- La lista de mensajes viaja entre agentes.
- Simple pero crece rápido (coste, drift).
- Resume o trunca antes de pasar.

## Workspace compartido

- Dict / objeto al que cada agente lee y escribe.
- Claves explícitas: research, draft, errors.
- Fácil de inspeccionar y debuggear.

## Payload tipado en handoff

- Pydantic model con solo lo que el siguiente agente necesita.
- Mejor opción a medio plazo.
- Fuerza diseño de interfaces.

Payloads pequeños y tipados > transcript completo. Tools compartidas: la misma list[Tool] se pasa a cada agente del crew.

# Cuándo NO ir multi-agente

Cuatro señales claras de que un solo agente con buenas tools será mejor.

## **Una tarea simple**

Si un único system prompt + 2-3 tools resuelve el problema, partir el agente solo añade puntos de fallo.

## **Acoplamiento fuerte**

Los sub-agentes necesitan saber siempre el resultado del otro: en realidad es un solo flujo, no dos.

## **Optimización prematura**

Dividir antes de tener un agente sólido convierte cada bug en un debug distribuido.

## **Sin evals**

Sin un eval set sólido, multi-agente solo amplifica los fallos del agente base.

Empieza con un agente con buenas tools. Añade más solo cuando haya un límite claro de delegación.

# Cuándo NO ir multi-agente

Cuatro preguntas. Si la respuesta es SÍ a 1-2, sigue con un agente.

1. ¿La tarea es un único objetivo coherente? → SÍ → 1 agente.
2. ¿Las subtareas son secuenciales y obvias? → SÍ → workflow + 1 agente + tools.
3. ¿Necesitas system prompts/personas/modelos distintos por subtarea? → SÍ → considera multi-agente.
4. ¿Las delegaciones cruzan dominios (billing vs técnico)? → SÍ → multi-agente.

Ante la duda, empieza con uno. Dividir luego es barato; juntar es caro.

Cinco preguntas clave. Responde las antes de comenzar la construcción.

**1. OUTPUT** — ¿Qué devuelve y en qué formato? (estructura tipada)

**2. OBJETIVO** — ¿Cuál es la meta del agente en una sola frase?

**3. INPUT** — ¿Qué información recibe? (formato, fuentes, frecuencia)

**4. TOOLS** — ¿Qué herramientas necesita? (mín. 1, máx. 4)

**5. GUARDRAILS** — ¿Qué validaciones de input y output?

## Workshop Agentes



# WORKSHOP: Prototipado de agentes con openAI agents SDK

## ARCHIVO DEL NOTEBOOK

1\_Agents.ipynb

### Objetivos de aprendizaje

En este notebook aprenderás:

- Qué son los agentes de IA y por qué importan en operaciones de seguridad residencial
- Los 4 conceptos fundamentales del OpenAI Agents SDK: Agents, Handoffs, Guardrails, Sessions
- Cómo construir agentes con tools, validación y memoria — sobre el caso del técnico de instalación
- Cómo trabajar con un asistente de código (vibe coding) sin perder control del agente

### Caso transversal

Asistente para el **técnico de campo** en una instalación Securitas Direct: responde dudas técnicas sobre paneles, sensores y procedimientos — y en Bloque B lo conectaremos con manuales reales y citas.

### Configuración para Jupyter Notebooks

**Importante:** Los Jupyter notebooks ya tienen un event loop de asyncio corriendo. Para usar `Runner.run_sync()` necesitamos instalar y configurar `nest_asyncio`:

```
[ ]: !pip install -q --upgrade openai-agents==0.4.1 nest_asyncio==1.6.0 pandas "typing_extensions>=4.13.0"

# ⚠️ Tras instalar/actualizar paquetes, REINICIA el kernel (Kernel → Restart Kernel)
# antes de ejecutar las siguientes celdas. typing_extensions ya estaba importado.

[ ]: # Configurar nest_asyncio para permitir event loops anidados en Jupyter
import nest_asyncio

nest_asyncio.apply()
```



Reto



# Construye tu agente

Asistente técnico de instalación: responde dudas con citas a manuales internos.

## **Caso**

Técnico en una instalación pregunta. El agente busca en manuales y responde con citas (fichero + página).

## **Stack**

Jupyter notebook + OpenAI Agents SDK + Chroma. Plantilla provista, celdas a rellenar.

## **Criterios mínimos**

Usa `search_kb` · output Pydantic con citas · al menos 1 guardrail · pasa 3 de 5 tests del eval set.

## **Formato**

individual · “*vibe coding*” permitido.

Todo listo en el repo del workshop. Solo rellena los TODOs.



Notebook plantilla

2\_Reto\_asistente.ipynb · 7 celdas TODO  
marcadas · imports y fixtures listos.



Manuales reales en data/manuals/



Eval set

tests/cases.yaml · 5 preguntas con keyword  
esperado · TODO 7 lo carga y ejecuta.



Tip

Empieza por TODO 1 (ingesta). Sin Chroma  
poblado el resto de tools no funciona.



# A TRABAJAR

`2_Reto_asistente.ipynb`

**Agentes en producción**



# Checklist de producción

Seis controles críticos para la transición de prototipo a entorno real.

✓ **Eval suite ( $\geq 80\%$  pass)**

Mide regresiones antes de cada deploy.

✓ **Observabilidad**

Trazas, costes y errores por sesión.

✓ **Path de escalado humano**

Traspaso fluido cuando el agente duda.

✓ **Guardarraíles input + output**

Ni entrada hostil, ni salida sin contrato.

✓ **Cap de coste por sesión**

Límite duro contra bucles infinitos.

✓ **Plan de rollback**

Versiones de modelo y prompt seguras.

Infraestructura gestionada de AWS para llevar agentes a producción sin montar la plataforma desde cero.

## Qué es

Runtime gestionado para agentes en producción.

## Por qué existe

Escalado, seguridad y operación sin montarlo todo.

## Cuándo usarlo

Cuando el agente ya funciona y toca producir.

## Qué no es

No reemplaza LangGraph ni OpenAI Agents SDK.

## Idea clave

Framework arriba. AgentCore debajo.

## Traducción

Menos plataforma propia.  
Más producto y evals.

# AgentCore: componentes clave

Cuatro piezas de infraestructura que suelen aparecer cuando un agente deja de ser demo.

## Runtime

Ejecuta agentes como servicios gestionados.

## Memory

Contexto entre turnos sin prompt-stuffing.

## Tool Gateway

Tools y APIs controladas, auditables.

## Identity

Permisos reales según usuario y acción.

## Encaje

La lógica va en LangGraph o Agents SDK.

## Regla práctica

Úsalo cuando toca operar, no prototipar.



A black and white illustration of a crowded room, possibly a lecture hall or a public meeting. The room is filled with people of various ages and genders, many of whom are looking towards the front. In the upper left, a chalkboard displays the text "Q&A" and "MICHIGAN". The illustration is framed by a thick, dark border. Overlaid on the center of the image is the text "Q&A" in a large, white, serif font.

Q&A

KAHOOT!

**Kahoot!**



**EBIS** Business  
Techschool

[www.ebiseducation.com](http://www.ebiseducation.com)

